

In Data Structures and Algorithms, an algorithm is:

A finite sequence of well-defined steps to solve a computational problem efficiently. In DSA, focus is not just *correctness* but also:

- Efficiency (Time Complexity)
- Memory Usage (Space Complexity)

Array in Data Structures and Algorithms

An array is a collection of elements stored in contiguous memory locations.

Access is done using index (position)

Index starts from 0

Basic Array Algorithms

We will cover the most important ones:

- Traversal
- Insertion
- Deletion
- Searching

1. Traversal Algorithm

Visit each element of array

Algorithm: TRAVERSE

Input: Array A, size n

Output: Display elements

Steps:

1. Start
2. For $i = 0$ to $n-1$
 → Print $A[i]$
3. Stop

Complexity:

- $O(n)$

2. Insertion Algorithm

Insert element at a given position

Algorithm: INSERT

Input: Array A, size n, position pos, element X

Output: Updated array

Steps:

1. Start
 2. If $\text{pos} < 0$ OR $\text{pos} > n$
→ Print "Invalid Position"
 3. For $i = n-1$ down to pos
→ $A[i+1] = A[i]$
 4. $A[\text{pos}] = X$
 5. $n = n + 1$
 6. Stop
- Shift elements right
 - Insert at required position

Complexity:

- $O(n)$

3. Deletion Algorithm

Delete element from a given position

Algorithm: DELETE

Input: Array A, size n, position pos

Output: Updated array

Steps:

1. Start
2. If $\text{pos} < 0$ OR $\text{pos} \geq n$
→ Print "Invalid Position"
3. For $i = \text{pos}$ to $n-2$
→ $A[i] = A[i+1]$
4. $n = n - 1$
5. Stop

- Shift elements left
- Remove element

Complexity:

- $O(n)$

4. Searching Algorithm (Linear Search)

Algorithm: SEARCH

Input: Array A, size n, element X

Output: Index or -1

Steps:

1. Start
2. For $i = 0$ to $n-1$
 → If $A[i] == X$
 → Return i
3. Return -1
4. Stop

Complexity:

- $O(n)$

Example

Array: [10, 20, 30, 40]

- Insert 25 at pos 2 → [10, 20, 25, 30, 40]
- Delete pos 1 → [10, 25, 30, 40]
- Search 30 → index = 2

Singly Linked List in Data Structures and Algorithms

A singly linked list is a collection of nodes where each node contains:

- Data
- Next pointer → points to next node

Last node points to NULL

Each node:

[DATA | NEXT]

INSERTION

1. Insertion at Beginning

Algorithm: INSERT_BEGIN

Steps:

1. Start
2. Create new node N
3. Set `N.data = X`
4. Set `N.next = HEAD`
5. Set `HEAD = N`
6. Stop

2. Insertion at End

Algorithm: INSERT_END

Steps:

1. Start
2. Create new node N, set data
3. If `HEAD == NULL`
→ Set `HEAD = N`, Stop
4. Set `TEMP = HEAD`
5. While `TEMP.next != NULL`
→ Move `TEMP = TEMP.next`
6. Set `TEMP.next = N`
7. Stop

3. Insertion at Position (Middle)

Algorithm: INSERT_POS

Input: Position `pos`

Steps:

1. Start
2. Create new node N
3. If `pos == 0`
→ Insert at beginning

4. Set `TEMP = HEAD`
5. Move to (pos-1) node
6. Set `N.next = TEMP.next`
7. Set `TEMP.next = N`
8. Stop

DELETION

1. Deletion from Beginning

Algorithm: DELETE_BEGIN

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. Set `TEMP = HEAD`
4. Set `HEAD = HEAD.next`
5. Delete `TEMP`
6. Stop

2. Deletion from End

Algorithm: DELETE_END

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. If only one node
→ Set `HEAD = NULL`, Stop
4. Set `TEMP = HEAD`
5. While `TEMP.next.next != NULL`
→ Move `TEMP`
6. Delete `TEMP.next`
7. Set `TEMP.next = NULL`
8. Stop

3. Deletion at Position

Algorithm: DELETE_POS

Input: Position `pos`

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. If `pos == 0`
→ Delete from beginning
4. Set `TEMP = HEAD`
5. Move to `(pos-1)` node
6. Set `DEL = TEMP.next`
7. Set `TEMP.next = DEL.next`
8. Delete DEL
9. Stop

SEARCHING ELEMENT

Algorithm: SEARCH

Input: Element x

Output: Position or Not Found

Steps:

1. Start
2. Set `TEMP = HEAD, pos = 0`
3. While `TEMP != NULL`
 - If `TEMP.data == x`
→ Return pos
 - Move `TEMP = TEMP.next`
 - Increment pos
4. Return "Not Found"
5. Stop

Complexity

- Insertion (beginning) → **O(1)**
- Insertion (end/position) → **O(n)**
- Deletion → **O(n)**
- Searching → **O(n)**

Doubly Linked List in Data Structures and Algorithms

A doubly linked list is a list where each node has:

- Prev pointer (points to previous node)
- Data
- Next pointer (points to next node)

First node's `prev` = `NULL`

Last node's `next` = `NULL`

Each node:

[`PREV` | `DATA` | `NEXT`]

INSERTION

1. Insertion at Beginning

Algorithm: `INSERT_BEGIN`

Steps:

1. Start
2. Create new node `N`, set data
3. Set `N.prev` = `NULL`
4. Set `N.next` = `HEAD`
5. If `HEAD != NULL`
→ Set `HEAD.prev` = `N`
6. Set `HEAD` = `N`
7. Stop

2. Insertion at End

Algorithm: `INSERT_END`

Steps:

1. Start
2. Create new node `N`, set data
3. If `HEAD == NULL`
→ Set `HEAD` = `N`, Stop
4. Set `TEMP` = `HEAD`
5. While `TEMP.next != NULL`
→ Move `TEMP` = `TEMP.next`
6. Set `TEMP.next` = `N`
7. Set `N.prev` = `TEMP`
8. Set `N.next` = `NULL`
9. Stop

3. Insertion at Position (Middle)

Algorithm: `INSERT_POS`

Input: Position `pos`

Steps:

1. Start
2. Create new node `N`
3. If `pos == 0`
→ Insert at beginning
4. Set `TEMP = HEAD`
5. Move to `(pos-1)` node
6. Set `N.next = TEMP.next`
7. Set `N.prev = TEMP`
8. If `TEMP.next != NULL`
→ Set `TEMP.next.prev = N`
9. Set `TEMP.next = N`
10. Stop

DELETION

1. Deletion from Beginning

Algorithm: DELETE_BEGIN

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. Set `TEMP = HEAD`
4. Set `HEAD = HEAD.next`
5. If `HEAD != NULL`
→ Set `HEAD.prev = NULL`
6. Delete `TEMP`
7. Stop

2. Deletion from End

Algorithm: DELETE_END

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. If only one node
→ Set `HEAD = NULL`, Stop
4. Set `TEMP = HEAD`

5. While `TEMP.next != NULL`
 → Move TEMP
6. Set `TEMP.prev.next = NULL`
7. Delete TEMP
8. Stop

3. Deletion at Position

Algorithm: DELETE_POS

Input: Position `pos`

Steps:

1. Start
2. If `HEAD == NULL` → Underflow
3. If `pos == 0`
 → Delete from beginning
4. Set `TEMP = HEAD`
5. Move to position `pos`
6. Set `TEMP.prev.next = TEMP.next`
7. If `TEMP.next != NULL`
 → Set `TEMP.next.prev = TEMP.prev`
8. Delete TEMP
9. Stop

SEARCHING ELEMENT

Algorithm: SEARCH

Input: Element `x`

Output: Position or Not Found

Steps:

1. Start
2. Set `TEMP = HEAD, pos = 0`
3. While `TEMP != NULL`
 - If `TEMP.data == x`
 → Return `pos`
 - Move `TEMP = TEMP.next`
 - Increment `pos`
4. Return "Not Found"
5. Stop

Complexity

- Insertion (beginning) $\rightarrow O(1)$
- Insertion (end/position) $\rightarrow O(n)$
- Deletion $\rightarrow O(n)$
- Searching $\rightarrow O(n)$

Stack Using Array

A stack is a LIFO (Last In First Out) data structure.
Last inserted element is removed first.

Basic operations:

- **Push** $\rightarrow$ Insert element
- **Pop** $\rightarrow$ Remove element

We use:

- Array: `STACK[MAX]`
- Variable: `TOP = -1` (initially empty)

PUSH Operation (Insert Element)

Problem:

Insert an element into stack

Algorithm: PUSH

Input: Element x

Output: Updated stack

Steps:

1. Start
 2. If `TOP == MAX - 1`
 $\rightarrow$ Print "Stack Overflow" and Exit
 3. Else
 $\rightarrow TOP = TOP + 1$
 $\rightarrow STACK[TOP] = x$
 4. Stop
- If stack is full $\rightarrow$ cannot insert (**overflow**)
 - Otherwise $\rightarrow$ increment TOP and insert element

POP Operation (Remove Element)

Problem:

Delete element from stack

Algorithm: POP

Output: Deleted element

Steps:

1. Start
 2. If $TOP == -1$
→ Print "Stack Underflow" and Exit
 3. Else
→ $X = STACK[TOP]$
→ $TOP = TOP - 1$
→ Return X
 4. Stop
- If stack is empty → cannot delete (**underflow**)
 - Otherwise → remove top element and decrease TOP

Example

Stack (size = 5):

- Push 10 → $TOP = 0$
- Push 20 → $TOP = 1$
- Push 30 → $TOP = 2$

Now Pop:

- Removes 30 → $TOP = 1$

Complexity

- Push → **$O(1)$**
- Pop → **$O(1)$**

Both operations are constant time

Queue Using Array

A queue is a FIFO (First In First Out) data structure.
First inserted element is removed first.

Basic operations:

- **Enqueue** → Insert element (rear)
- **Dequeue** → Remove element (front)
- **IsFull** → Check overflow
- **IsEmpty** → Check underflow

We use:

- **Array:** `QUEUE [MAX]`
- `FRONT = -1, REAR = -1` (initially empty)

ENQUEUE Operation (Insert)

Algorithm: ENQUEUE

Input: Element x

Output: Updated queue

Steps:

1. Start
 2. If `REAR == MAX - 1`
→ Print "Queue Overflow" and Exit
 3. If `FRONT == -1`
→ Set `FRONT = 0`
 4. `REAR = REAR + 1`
 5. `QUEUE [REAR] = X`
 6. Stop
- Insert element at **rear**
 - If full → overflow
 - If first element → initialize FRONT

DEQUEUE Operation (Delete)

Algorithm: DEQUEUE

Output: Deleted element

Steps:

1. Start
 2. If $\text{FRONT} == -1$ OR $\text{FRONT} > \text{REAR}$
→ Print "Queue Underflow" and Exit
 3. $X = \text{QUEUE}[\text{FRONT}]$
 4. If $\text{FRONT} == \text{REAR}$
→ Set $\text{FRONT} = \text{REAR} = -1$
Else
→ $\text{FRONT} = \text{FRONT} + 1$
 5. Return X
 6. Stop
- Remove element from **front**
 - If empty → underflow
 - Reset when last element removed

ISFULL Operation

Algorithm: ISFULL

Steps:

1. Start
2. If $\text{REAR} == \text{MAX} - 1$
→ Return TRUE
3. Else
→ Return FALSE
4. Stop

ISEMPTY Operation

Algorithm: ISEMPTY

Steps:

1. Start
2. If $\text{FRONT} == -1$ OR $\text{FRONT} > \text{REAR}$
→ Return TRUE
3. Else
→ Return FALSE
4. Stop

Example

Queue (size = 5):

- Enqueue 10 $\rightarrow$ FRONT=0, REAR=0
- Enqueue 20 $\rightarrow$ REAR=1
- Enqueue 30 $\rightarrow$ REAR=2

Dequeue:

- Removes 10 $\rightarrow$ FRONT=1

Complexity

- Enqueue $\rightarrow \mathbf{O(1)}$
- Dequeue $\rightarrow \mathbf{O(1)}$
- IsFull / IsEmpty $\rightarrow \mathbf{O(1)}$